Lecture 8

Virtual Memory & Memory Management

Mingyu Gao

gaomy@tsinghua.edu.cn

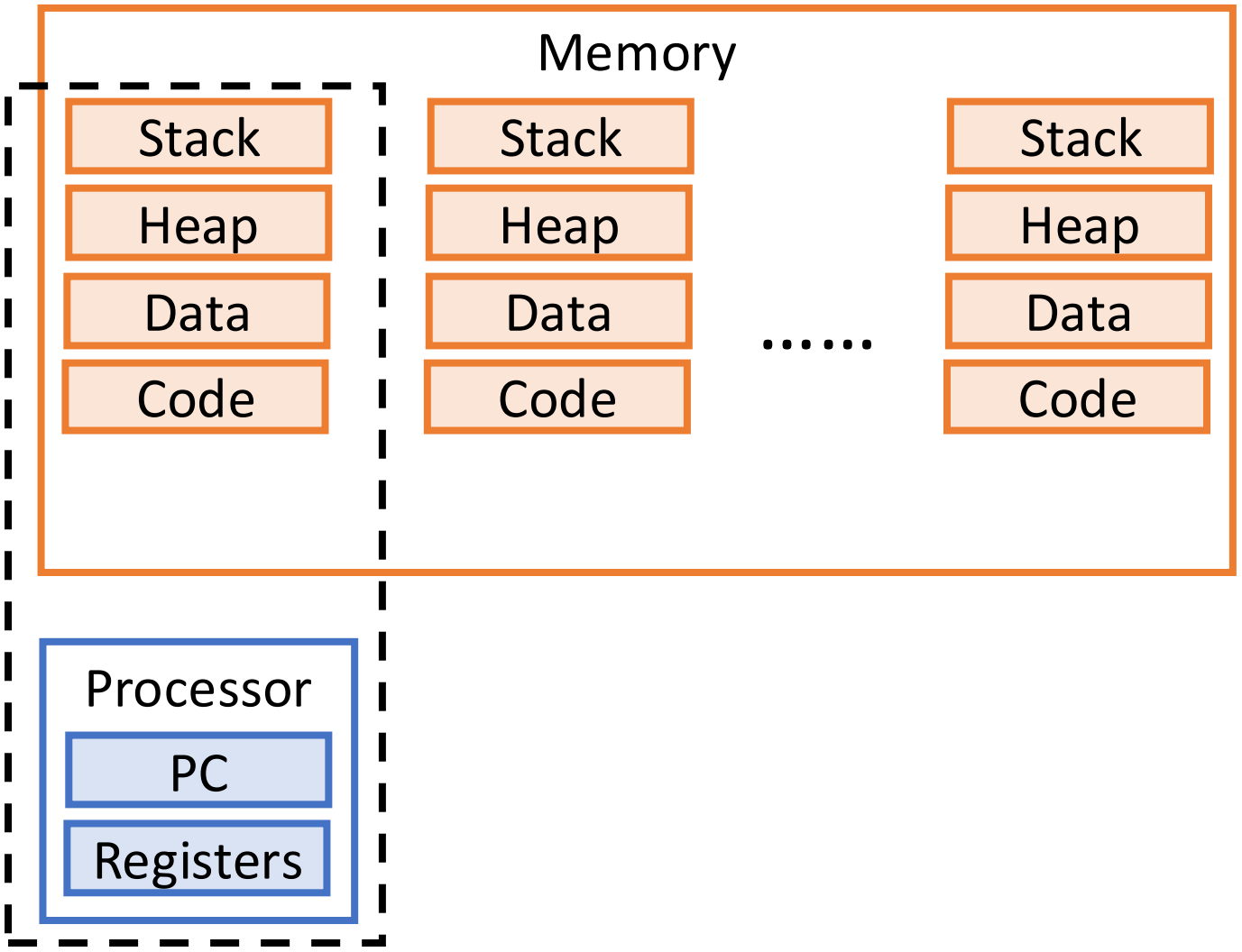
IIIS, Tsinghua

Review: Where We Are Now

- Hardware architectures of Von Neumann
  - Processors and assembly languages
  - Memory hierarchies and caches
  - Input/output devices
- Operating systems: manage hardware resources
  - Processes and threads, multi-threading programming
    - Manage processors
  - Virtual memory
    - Manage memory
  - File systems
    - Manage some I/O devices like disks/SSDs
- Operating systems: protection

← *Today's topic*

Review: Address Space of A Process

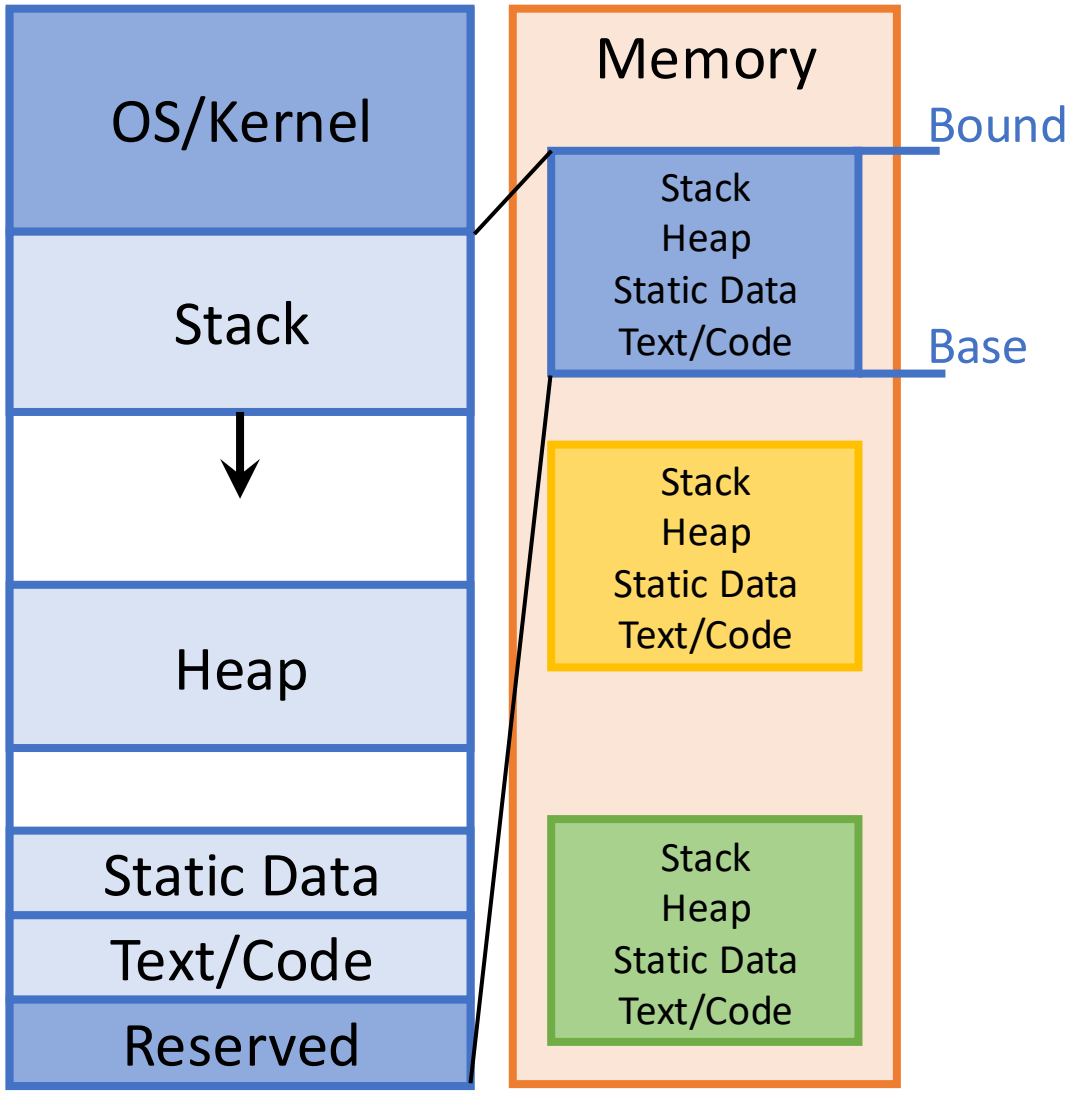


- Each process has one or more threads and one address space
- Address spaces of different processes are all mapped to and sharer the memory
- Address spaces must be protected and isolated from each other

Review: Simple Base and Bound (B&B)

- Base and bound (B&B)
- The entire physical memory is split into contiguous ranges that are assigned to different processes

- Drawbacks
  - Restricted to contiguous address ranges
  - Difficult to extend dynamically
  - Difficult to support shared regions across address spaces



Virtual Memory

- Virtual memory mechanism provides fine-grained and dynamic management of address spaces
  - Address space of each process does not have to be physically contiguous
  - Address space size can extend/shrink dynamically at runtime
  - Processes can share part of their address spaces with each other

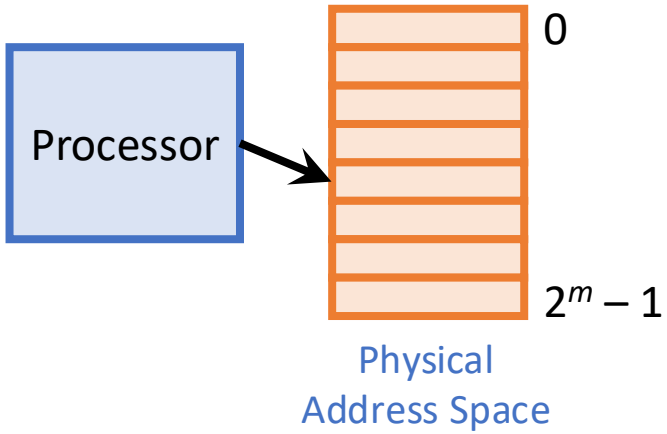
- Virtual memory is used in all modern servers, laptops, smart phones, …
  - Except for extremely “simple” systems like embedded microcontrollers
- One of the greatest ideas in computer science: indirection

Virtual and Physical Address Spaces

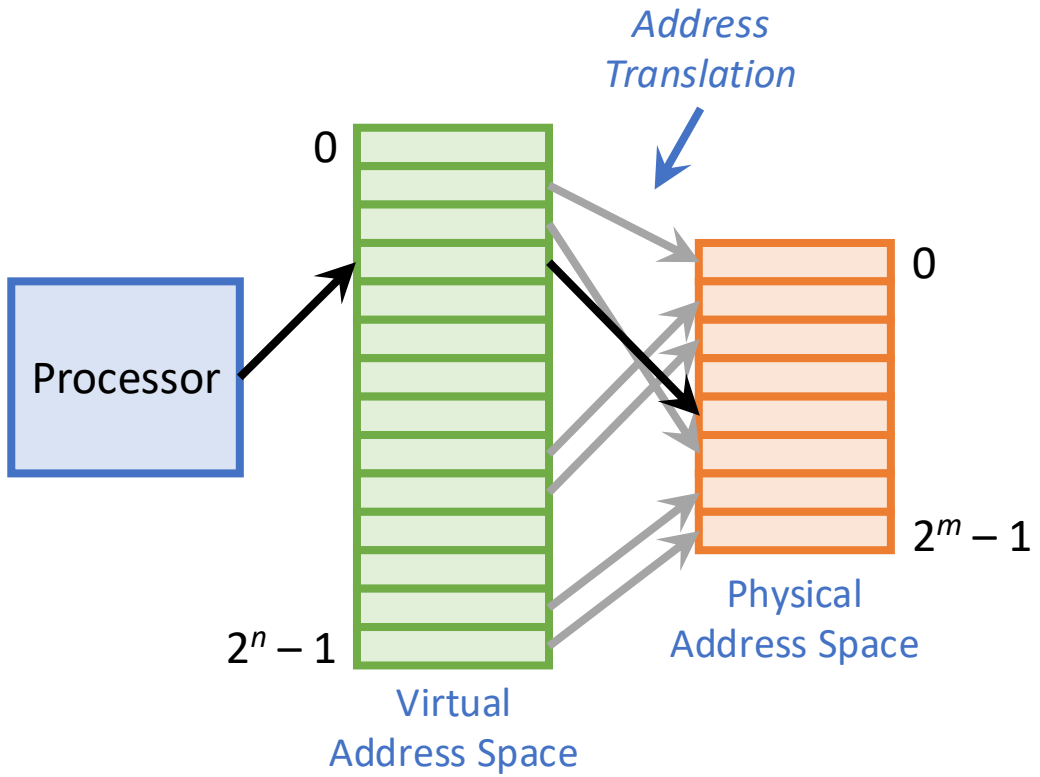
- Each process has its own virtual address space
  - Contiguous and linear, i.e., from 0 to $N = 2^n$ (n is 32-bit or 48-bit)
  - Virtual address space provide the desired exclusive illusion to process
- The hardware system has a single physical address space
  - Also contiguous and linear, from 0 to $M = 2^m$, but M is limited by actual main memory (e.g., DRAM) capacity, e.g., 32 GB, 64 GB
- All virtual address spaces are mapped to the physical address space in a flexible way, in the unit of pages
  - Each page is typically 4 kB, but sometimes can also be 2 MB or 1 GB
- Address translation: virtual address → physical address

Flexible Virtual-to-Physical Mapping

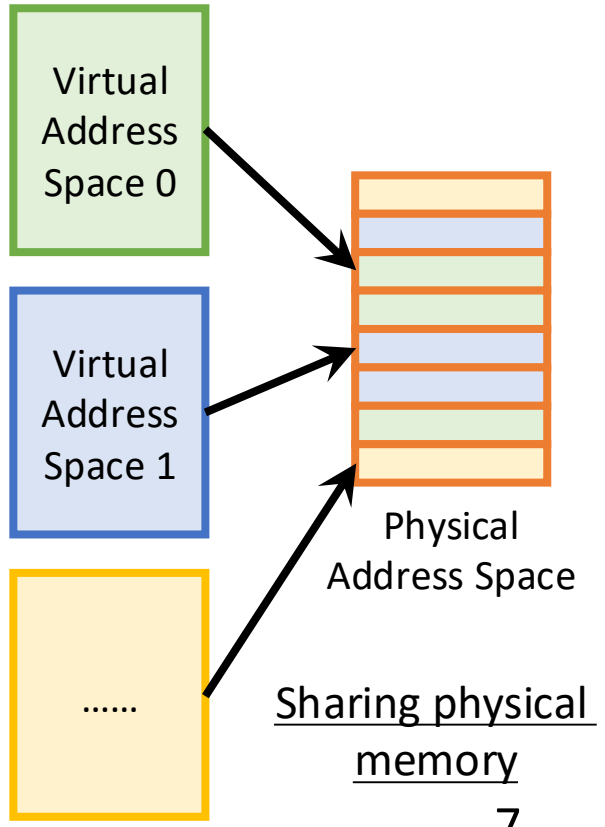
- Contiguous virtual pages may not be contiguous in physical address space
- Contiguous physical pages may not belong to the same virtual address space
- Not all virtual pages are mapped to physical space: unmapped pages



Without virtual memory



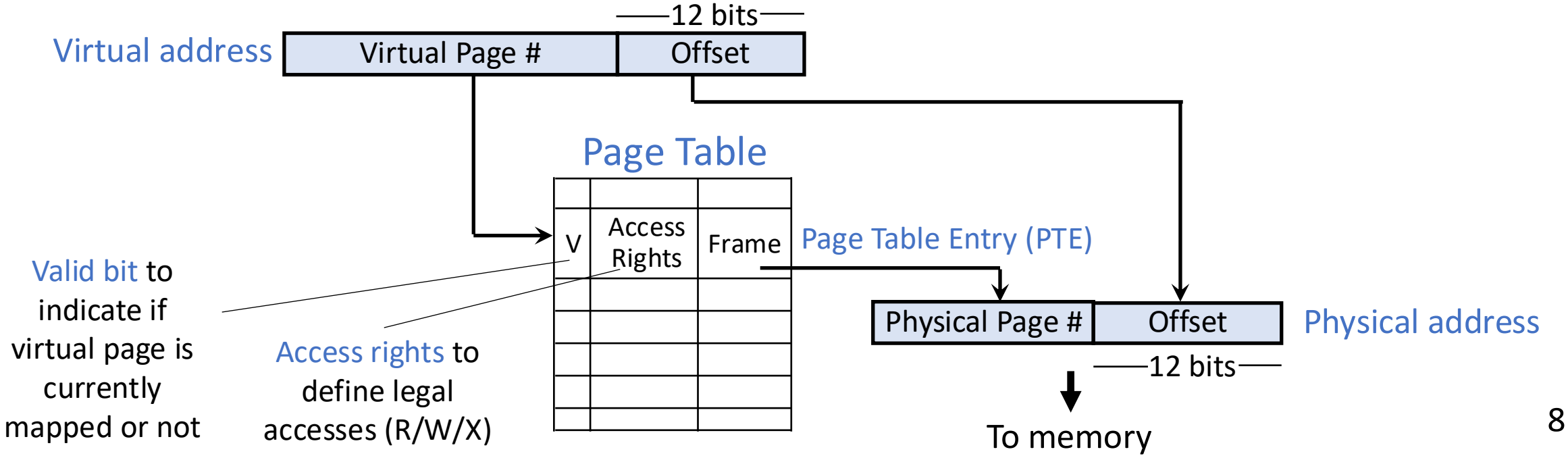
With virtual memory



Sharing physical memory

Address Translation with Page Table

- Page table: translate virtual address to physical address
  - Each process has its own page table; managed by OS; used by hardware
  - Page table is stored in main memory as an OS data structure
  - One page table entry (PTE) per virtual page (lower address bits are not translated)
  - Each PTE includes: valid bit, physical page number (a.k.a., frame number), metadata (e.g., R/W/X permissions)



Why Virtual Memory?

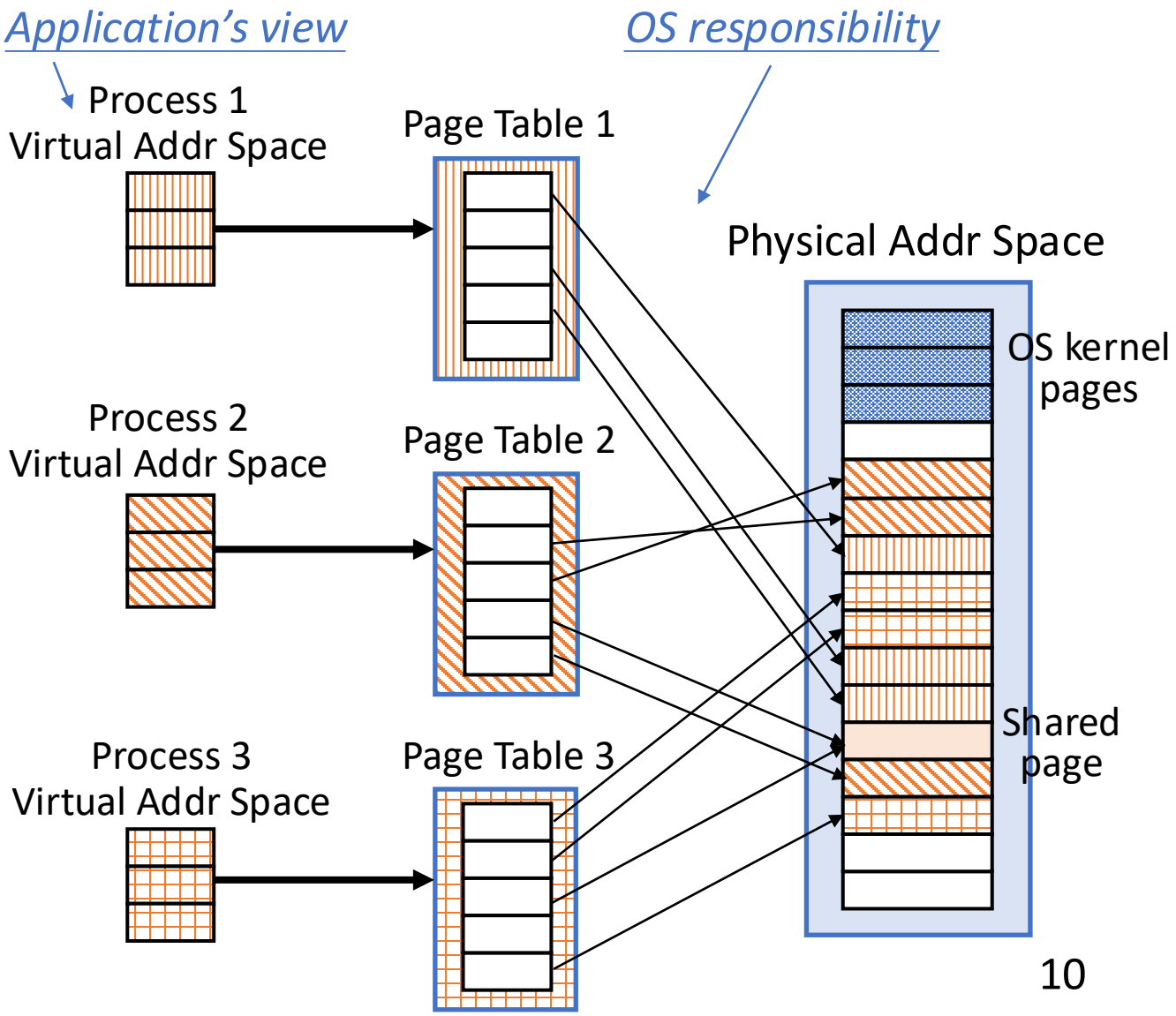
- Virtual memory as a tool for memory management
  - Each process gets a linear (virtual) address space that it can exclusively access
  - User processes do not deal with physical address mapping; it is the OS job!

- Virtual memory as a tool for memory protection
  - One process cannot directly access other's data through its virtual address space
  - User processes cannot access kernel if not mapped to its virtual address space
  - Each page has permission information stored in page table

- Virtual memory as a tool for data caching
  - Use DRAM as a cache for part of a virtual address space; only the data that need to be accessed are mapped to physical DRAM

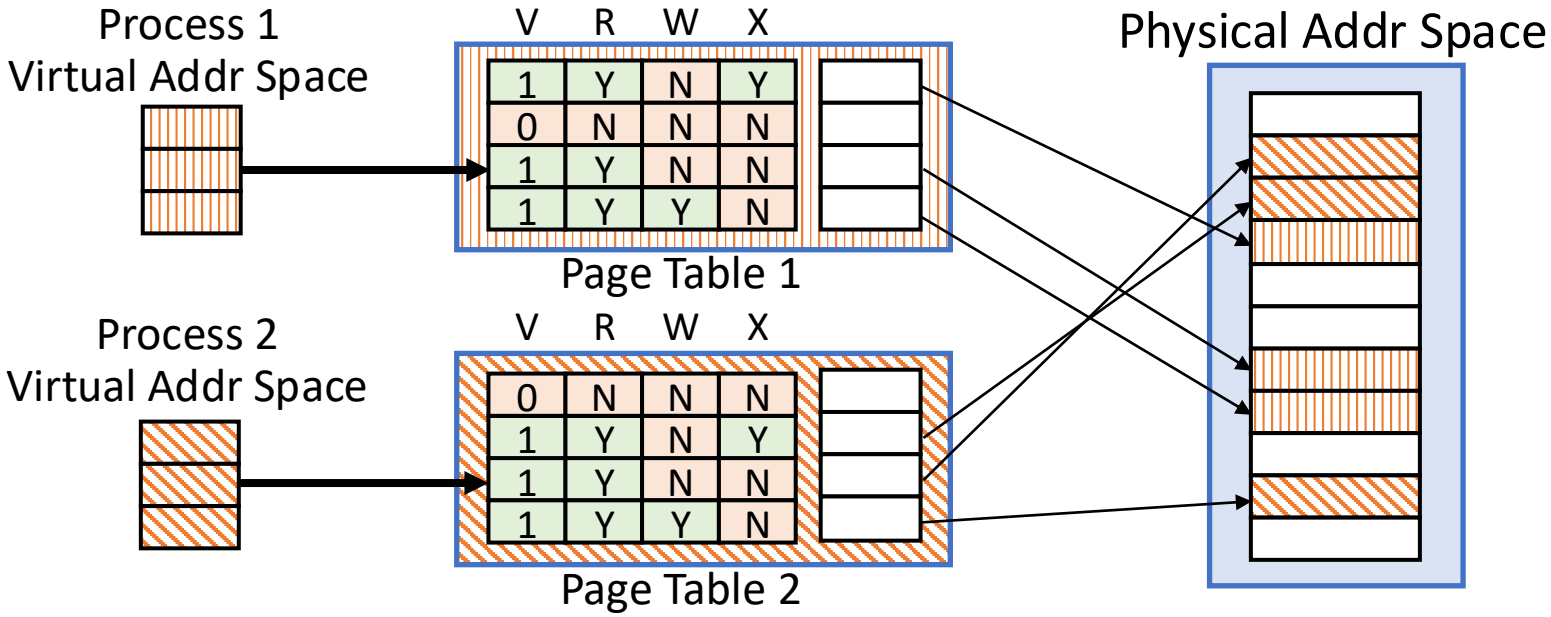
Virtual Memory for Memory Management

- Each process has its own virtual address space. It can view memory as the same simple linear array as before
- Allocate & map pages in a virtual space: find a physical frame, set up page table entry
  - Q: What if no free physical frame?
- Also easy to share pages across processes
  - Map their virtual pages to the same physical page



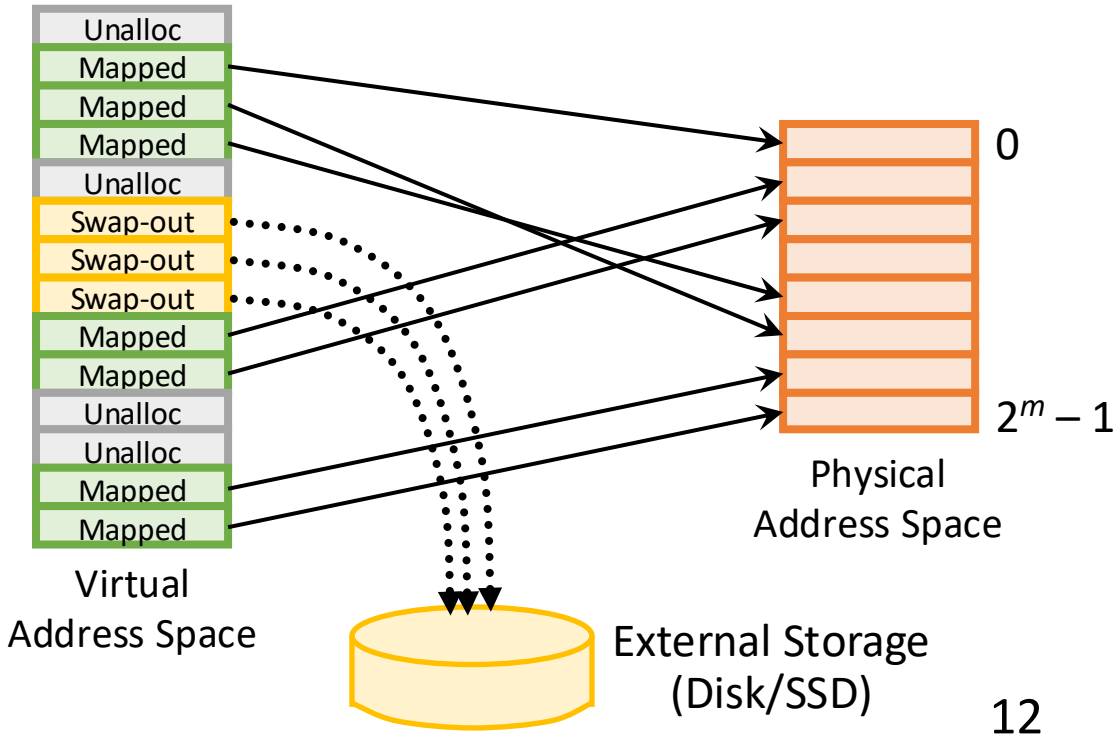
Virtual Memory for Memory Protection

- Process cannot access physical memory not mapped to its virtual space
  - OS manages the page table, and ensures that other processes' data and kernel data are not mapped to the process virtual address space
- PTE has permission bits: Read, Write, eXecute
  - Hardware checks these bits on each access when doing translation
  - If no permission, raise an exception to the user process



Virtual Memory for Data Caching

- For a process, usually only a subset of its virtual pages are allocated
  - Virtual space size ($N = 2^n$ bytes) is huge; e.g., $N = 2^n = 2^{48}$ bytes when $n = 48$ bits
- Only a subset of allocated virtual pages are mapped
  - Physical space size ($M = 2^m$ bytes) is limited; e.g., M = 64 GB to 512 GB, and shared by many processes
  - Main memory (DRAM) only caches the most frequently accessed data
- Allocated but unmapped pages are swapped out to external storage, e.g., disks or SSDs

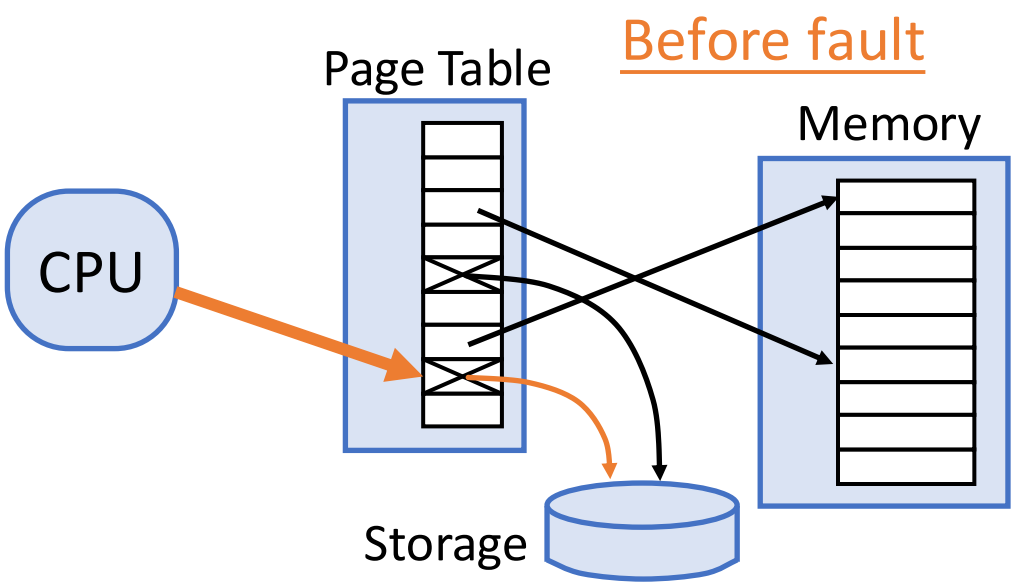


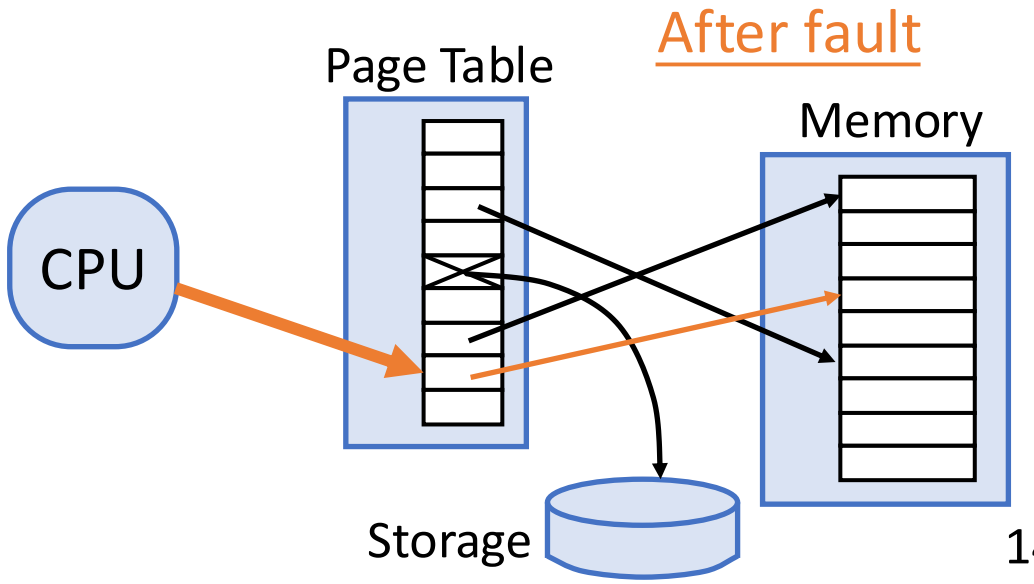
Virtual Memory as Cache and Demand Paging

- DRAM is like a software-managed cache of disks/SSDs
  - The cache blocks are pages (4 kB) rather than processor cachelines (64 B)
  - OS determines what data to put in DRAM and what data to swap out
  - Q: Is this cache fully associative, set associative, or direct mapped?
- Page table, managed by OS, always knows the data location
  - A valid physical page number in PTE: page hit in DRAM
  - A location on disk, i.e., allocated but not mapped (swapped-out) page: page fault
    - Page fault is a type of exception, raised when hardware finds a page miss in DRAM
  - Non-existing locations: unallocated
- Demand paging: if a page fault occurs, the page is paged in on demand, by the page fault handler in OS kernel

Page Fault Details

- Invoke OS page fault handler to move data from storage into memory
  - Similar to a cache miss, but handled in software
  - Choose replacement, swap out the victim, page in the page, update page table
- The process suspends until resolved, and retries the faulted load/store
  - Page fault handling is extremely slow!
  - Other processes run without impacted





Putting Altogether: Translation Process Details

- OS manages page table by allocating physical space to virtual space
- Hardware looks up page table to translate virtual to physical address

- If it is a valid page (V == 1)
  - Check access permissions (R, W, X) against access type
  - If allowed, translate to physical address and access memory
  - Otherwise, generate a permission fault, i.e., an exception, handled by OS

- Otherwise, an invalid page (V == 0), i.e., page is not currently in memory
  - Generate a page fault, i.e., an exception, handled by OS
  - If it is due to program errors, e.g., unallocated: terminate process
  - If page is on external storage: refill & retry, i.e., demand paging

Locality, Again!

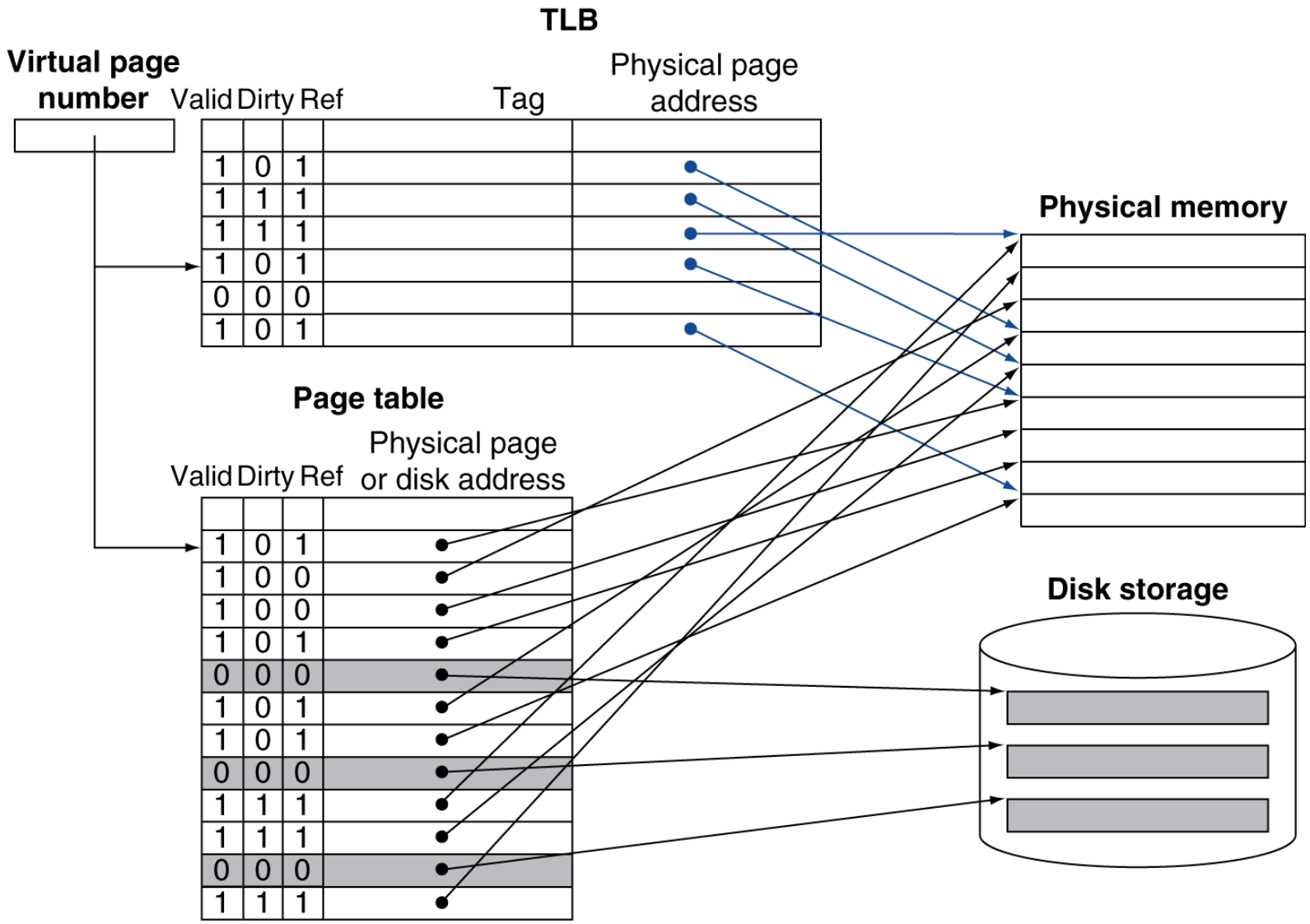
- Virtual memory seems to be terribly inefficient
  - Every access needs a translation before going to memory
  - A page fault to bring data from disk is very slow
- But it works because of locality!

- At any time, the program tend to access a limited set of active pages, called its working set
  - If this working set fits in main memory, very few page faults will occur
  - Otherwise, thrash! Buy your computer more memory!
- We can also cache the translation on-chip – TLB

Speeding Up Translation With TLB

- Translation Look-aside Buffer (TLB) = a hardware cache just for address translation entries, i.e., PTE
- Each TLB entry
  - Data: a page table entry (PTE), including physical page number and R/W/X bits
  - Valid bit, tag, etc. of itself
- Key idea: locality in accesses → locality in translations
- To distinguish
  - Main memory as a cache for disks: data
  - TLB as a cache for page table: translation

TLB Organization



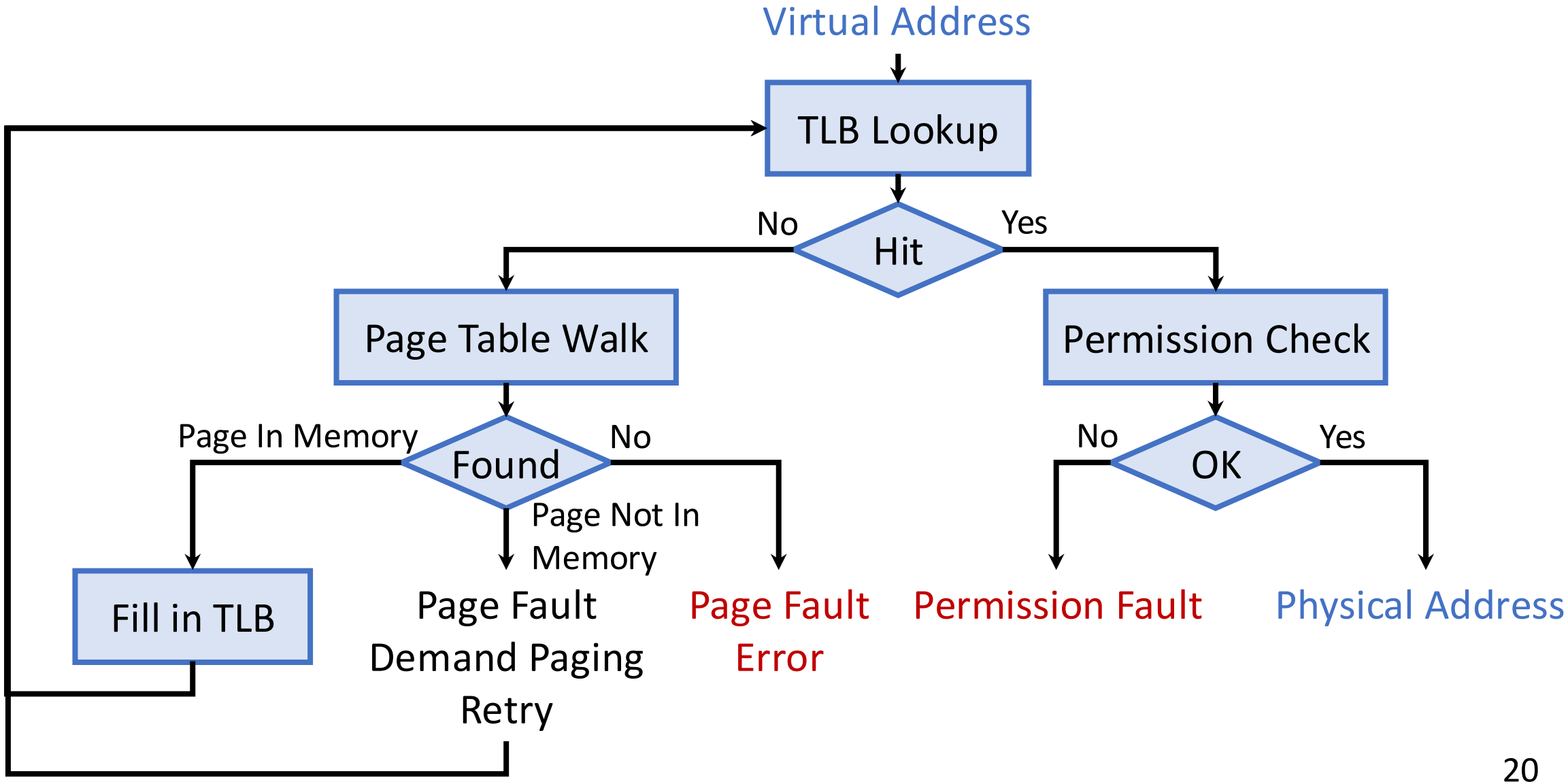
TLB Miss Handling: Page Table Walk

- If we miss in TLB, access PTE in memory; this is called page table walk
- Page table walk could be handled in hardware (e.g., x86)
  - Can get complex for more complicated page table structures
- Or in software
  - Raise a special exception, to use a special handler

- If page is in memory, copy PTE into TLB and retry
- If page is not in memory, resolve the page fault first, then fill in TLB

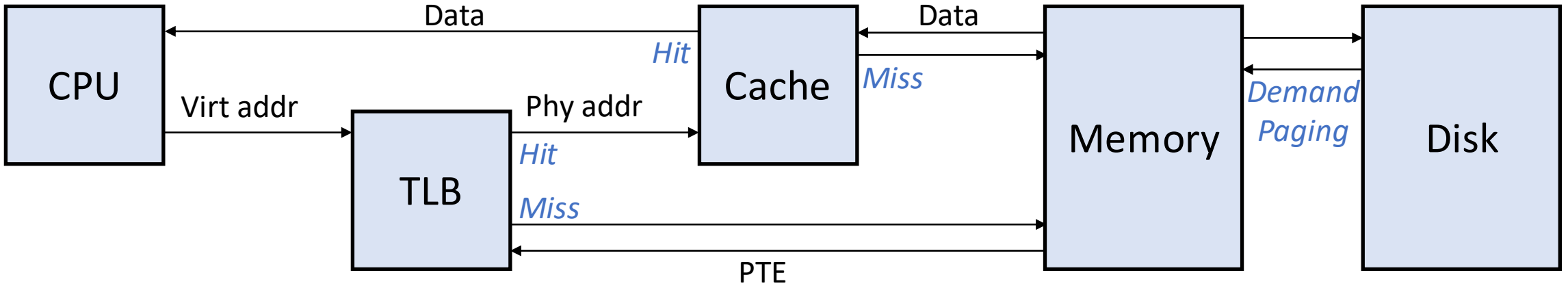
- Note the difference between a page table walk and a page fault
  - Miss in main memory as a cache for disks: page fault
  - Miss in TLB as a cache for page table: page table walk

Virtual Memory: Complete Flow



Virtual Memory & Cache Hierarchy

- First access TLB to translate virtual address to physical address
  - May cause page table walk and/or page fault
- Then access caches and DRAM using physical address
  - Cache is indexed and tagged both using physical address
  - Q: Can we use virtual addresses in data caches?



Problem of Page Table Size

- Page table size is proportional to virtual address space size
  - Virtual address space is much larger than typical physical address space!
  - x86-64 example: virtual addresses are 48 bits, pages are 4 kB, 64-bit PTE
    - Total number of pages: 2^{48} / 4096 = 2^{36} = 64 billion!
    - Total page table size is therefore 2^{36} × 64 bits = 512 GB!
  - And worse, what about running multiple processes?

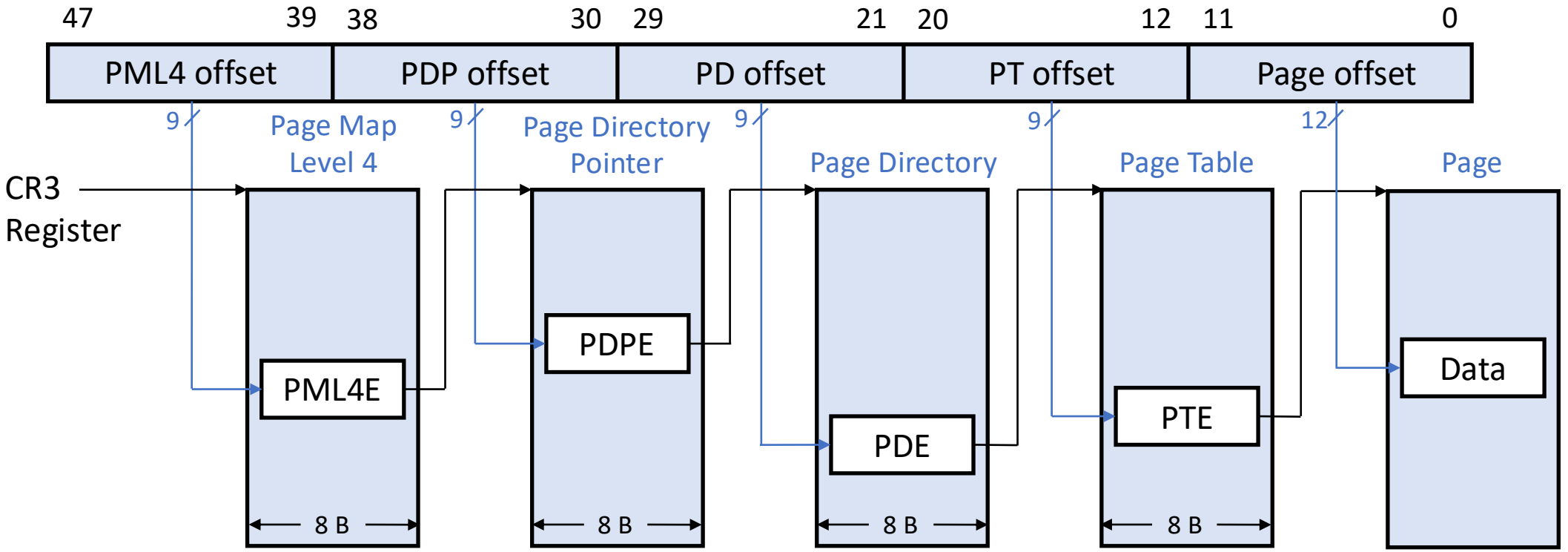
- Why is this a problem?
  - Page table must also reside in memory, not disks/SSDs, to support fast accesses

Multi-Level Page Table

- But, only a small fraction of pages are actually allocated, i.e., sparsity!
- Ideally, we only need PTEs of allocated pages, not all virtual pages
- We know how to store sparse data --- using an index!
  - E.g., B+tree, hash table, …
  - An index takes much smaller space than a full table when storing sparse data
- The most commonly used index for page table: radix tree
  - See appendix for data structure details
- Radix-tree-based page table is also known as multi-level page table
  - Only top level must be resident in memory
  - Remaining levels can be in memory or on disk
  - Or unallocated if corresponding ranges of the virtual address space are not used

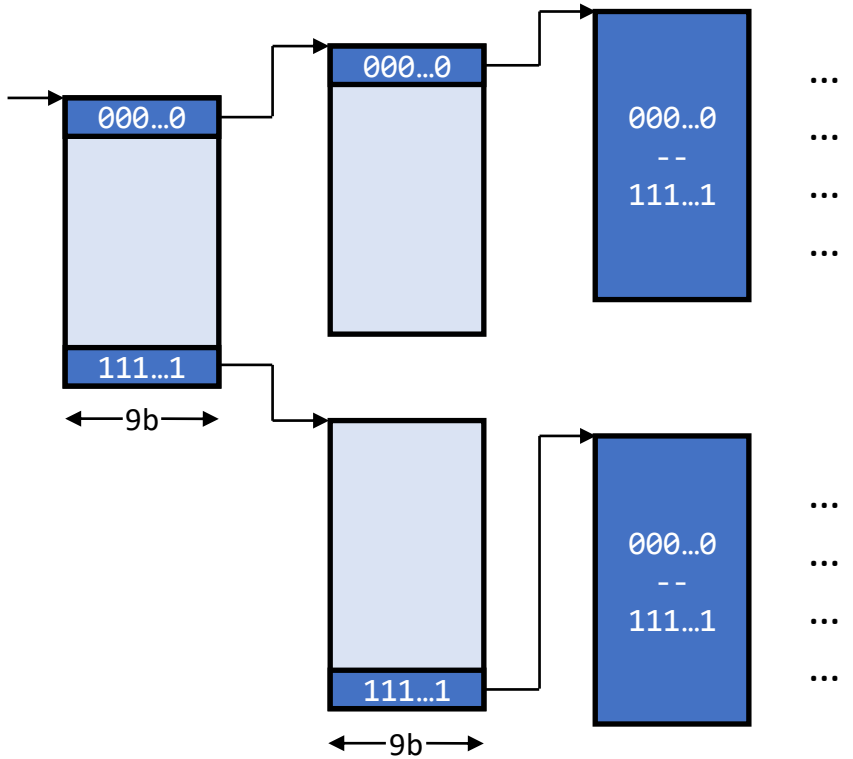
Example: Intel x86-64 Four-Level Translation

- x86-64 four-level page table is a 512-ary radix tree, with span of 9 bits



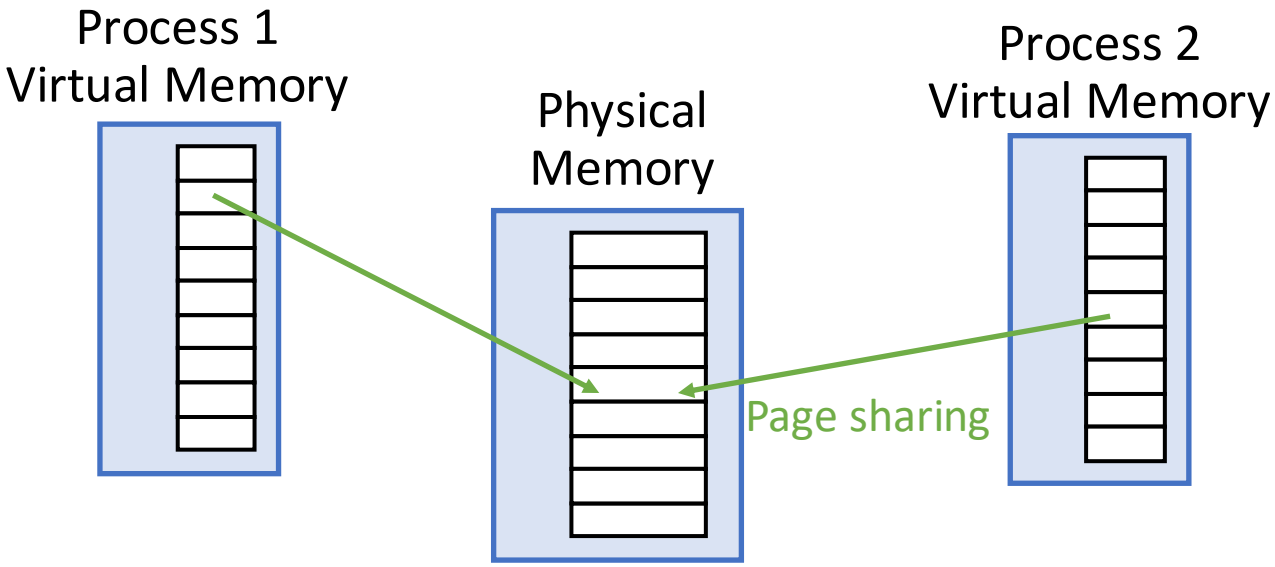
How Multi-Level Page Table Saves Space

- If only some virtual address ranges are sparsely allocated, the subtrees for the non-allocated ranges can be empty
- Example
  - Assume only following virtual addresses are valid:
    `0xFFFF C000 0000` … to `0xFFFF FFFF FFFF` …,
    `0x0000 0000 0000` … to `0x0000 3FFF FFFF` …
  - The first 18 bits are either all 0s or all 1s
  - Only part of page table shown in right is needed
- Q: What are the disadvantages?



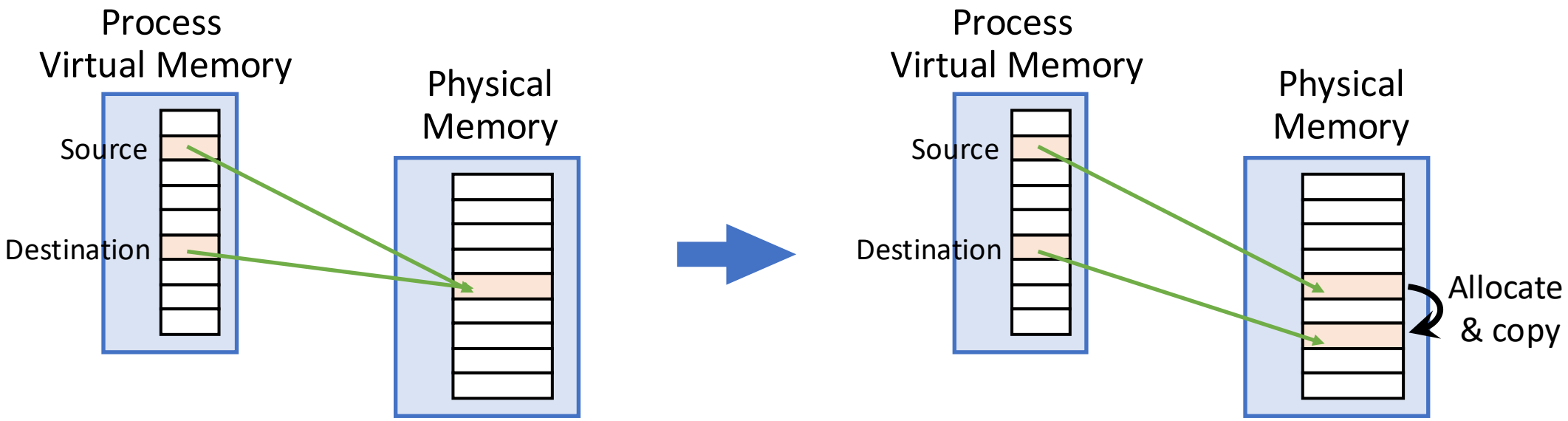
Inter-Process Sharing

- Page sharing: different processes can share a page by setting their virtual addresses to point to the same physical address
  - Process 1 first maps the shared physical page to its virtual address space
  - Process 2 then maps the same physical page to its virtual address space
  - The virtual addresses in the two processes could be different



Copy-on-Write (CoW)

- When copying pages within one process or across two processes
- Copy-on-Write (CoW)
  - When copying, just make destination virtual address point to the same physical page as source virtual address, and mark the page as write-protected in PTE
  - When writing to either virtual address, trigger a permission fault and do actual copy, i.e., allocate a new physical page



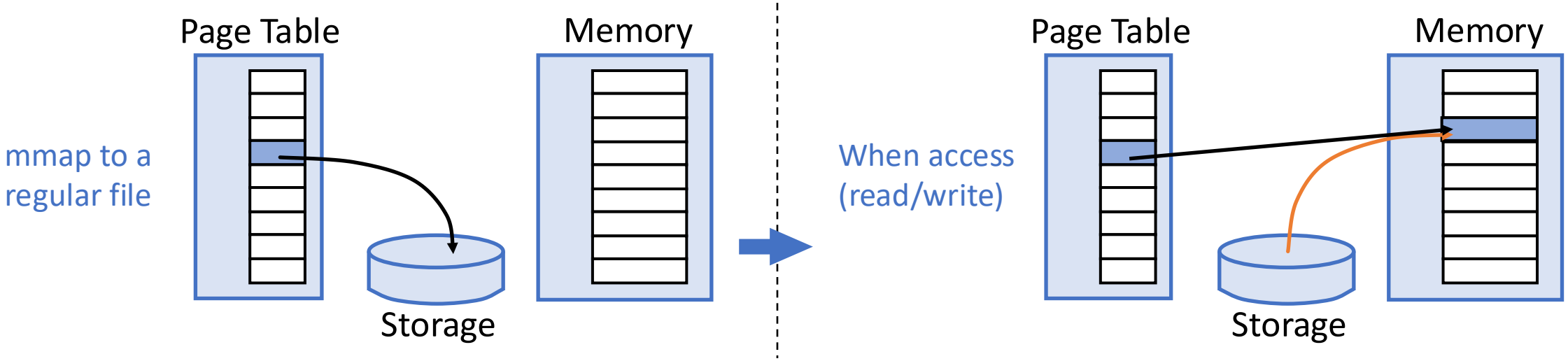
Memory Mapping

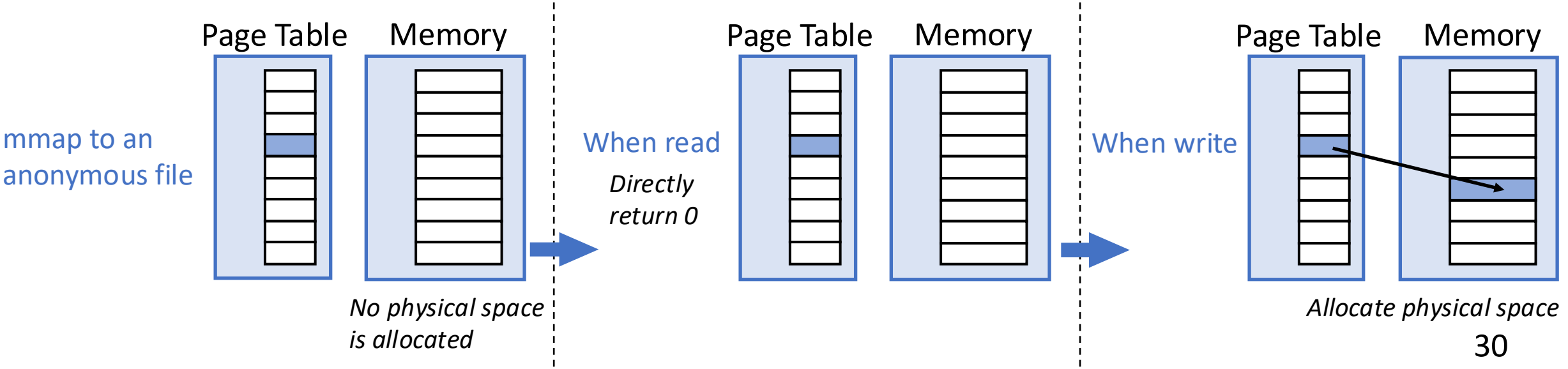
Memory Mapping

- Memory mapping: associate a virtual memory area with a file object
  - Get initial data from this file into memory: regular file on disk
  - Swap the physical memory content from/to this file: special swap file
  - For newly allocated space initialized to all zero values: special anonymous file
- A unified way to deal with all above cases, i.e., all data sources to the memory area are treated as files

- Lazy data transfer: at mapping time, OS only sets up PTE in page table; wait until the first page fault (when data are actually accessed), and rely on demand paging to get data into memory
  - Read data from the regular file or swap file
  - For anonymous file, physical page is only allocated at the time of the first write; before that, just return 0 for read accesses

Memory Mapping to Regular/Anonymous Files





Revisit fork and execve Syscalls

- Virtual memory and memory mapping allow `fork` syscall to create private address space for each new process
  - Recall: child gets an identical but separate copy of the parent's address space
  1. Create exact copies of the page table, but not yet copy data pages; i.e., CoW (mark each PTE in both page tables as write-protected)
  2. At this point, both processes have the identical contents in their address spaces
  3. Subsequent writes to either address space use CoW to create separate pages
- To load and run a new program by `execve`, etc.
  1. Free the old page table and initialize a new page table
  2. Memory map program and data
     - Program and statically initialized data are backed by the executable file
     - Stack is backed by anonymous files
  3. Set PC to the entry point of the program

User-Level Memory Mapping

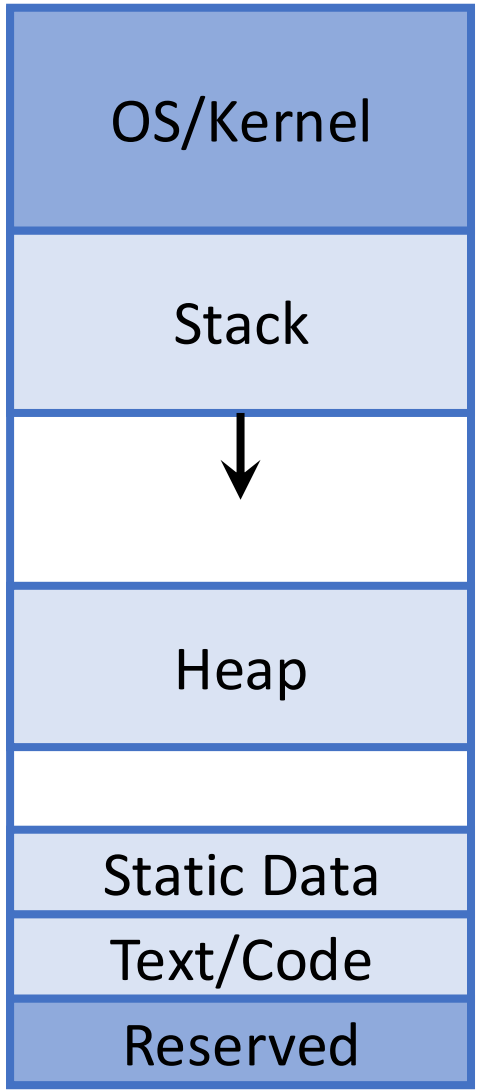
- mmap and mumap syscall
  - Used for accessing files as well as allocating new virtual address ranges (anonymous file)
  - `void* mmap(void* addr, size_t length, int prot, int flags, int fd, off_t offset)`
    - Map `length` bytes from the offset `offset` of the file specified by the descriptor `fd`, preferably to virtual address `addr`
    - `addr` is only a hint; it could also be NULL to let kernel choose the virtual address
    - `prot`: desired protection, e.g., `PORT_READ`, `PORT_WRITE`, …
    - `flags`: `MAP_ANON`, `MAP_PRIVATE`, `MAP_SHARED`, …
    - Return the mapped virtual address (may not equal to `addr`), or `MAP_FAILED` on error
  - `int munmap(void* addr, size_t length)`

```
int fd = open(filename, O_RDONLY);
if (fd < 0) { printf("file open error!"); exit(-1); }
struct stat finfo;
if (fstat(fd, &finfo) < 0) { printf("file stats error!"); exit(-1); }
void* buf = mmap(0, finfo.st_size, PROT_READ, MAP_PRIVATE | MAP_POPULATE, fd, 0);
if (buf == MAP_FAILED) { printf("mmap error!"); exit(-1); }
if (munmap(buf, finfo.st_size) < 0) { printf("munmap error!"); exit(-1); }
```

Dynamic Memory Allocation

Dynamic Memory Allocation

- A program can dynamically allocate memory at runtime
  - Useful when data structure sizes are only known at runtime
  - E.g., `malloc()` function in C, `new` statement in C++
- The dynamic allocator manages an area of the process virtual address space, called the heap
- Allocator vs. `mmap` syscall
  - User program uses `malloc()` or `new` statement to ask allocator for a space of any size for its data structure, within the heap and within the process' already allocated virtual space
  - Internally, allocator uses `mmap` and `sbrk` syscalls to extend the virtual address space of the process to grow the heap size in unit of pages if needed



Allocator API

- `void* malloc(size_t size)`
  - Get a pointer to a memory space of at least `size` bytes
  - Other variants: `calloc`, `realloc`, …
- `void free(void *p)`
  - Give up the memory space pointed by `p`, previously from `malloc/calloc/…`

```
// Allocate n ints from the heap
int* p = (int*)malloc(n * sizeof(int));
if (p == NULL) { printf("malloc error"); exit(-1); }

// Operate on the allocated data
for (int i = 0; i < n; i++) p[i] = i;

// Give back allocated block to the heap
free(p);
```

- C++ `new` and `delete` statements are similar

External Fragmentation

- Naïve allocator policy like first-fit may cause fragmented memory space
  - I.e., enough aggregate free space, but no single contiguous free space

```
p1 = malloc(4);
```

```
p2 = malloc(5);
```

```
p3 = malloc(6);
```

```
free(p2);
```



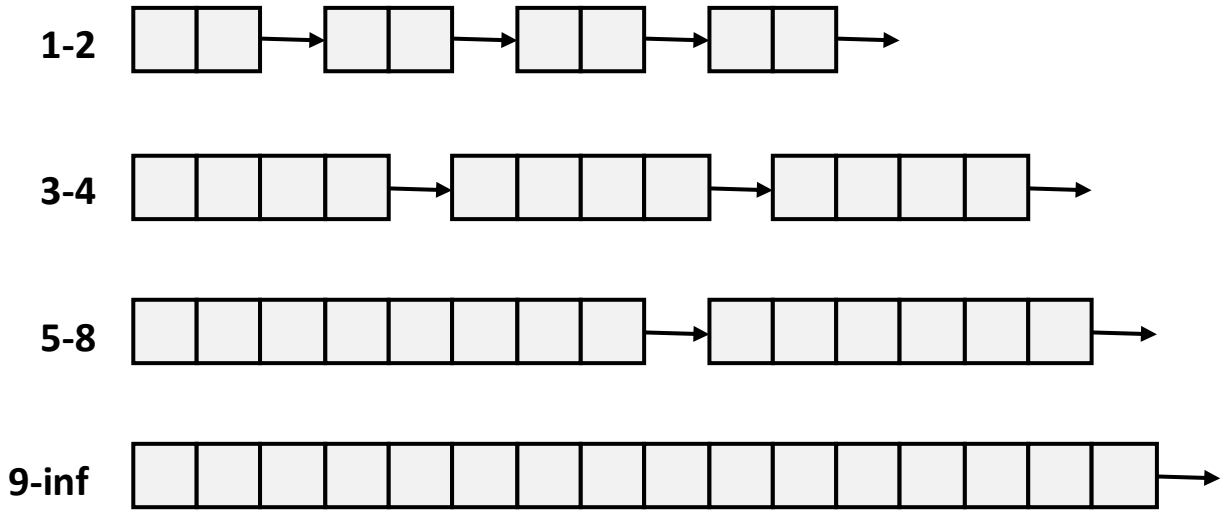
```
p4 = malloc(6);
```

Oops!

- Q: What would be a better policy?

Segregated Free Lists

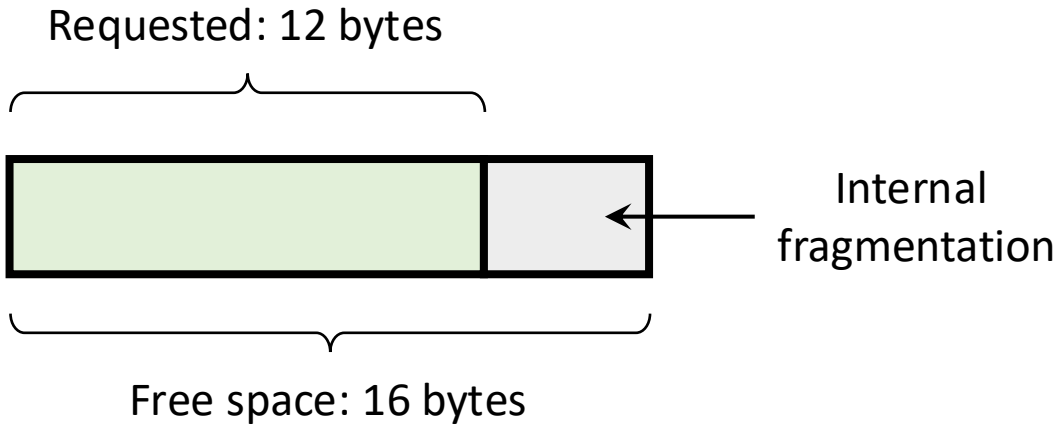
- Organize free spaces into multiple linked lists for different sizes
  - E.g., one list for each power-of-two size



- To allocate a block of *n* bytes
  - Search the free list of size $m > n$
  - If found a free space, use the first *n* bytes --- limited fragmentation ($m - n$ bytes)
  - If no free space is found, try next larger size
  - If no space is available in all lists, grow heap from OS

Internal Fragmentation

- Internal fragmentation happens when the requested size is smaller than the allocated free space, and the remaining space is wasted
- Caused by
  - Padding for alignment
  - Allocator policy, e.g., segregated lists
  - Metadata store, e.g., size, status
  - ……



- One solution: append remaining space of ($m - n$) to its corresponding free list, which can be used for a future small-size request

- Good allocators should be fast, and minimize both external and internal fragmentation

Garbage Collection

Implicit Memory Management

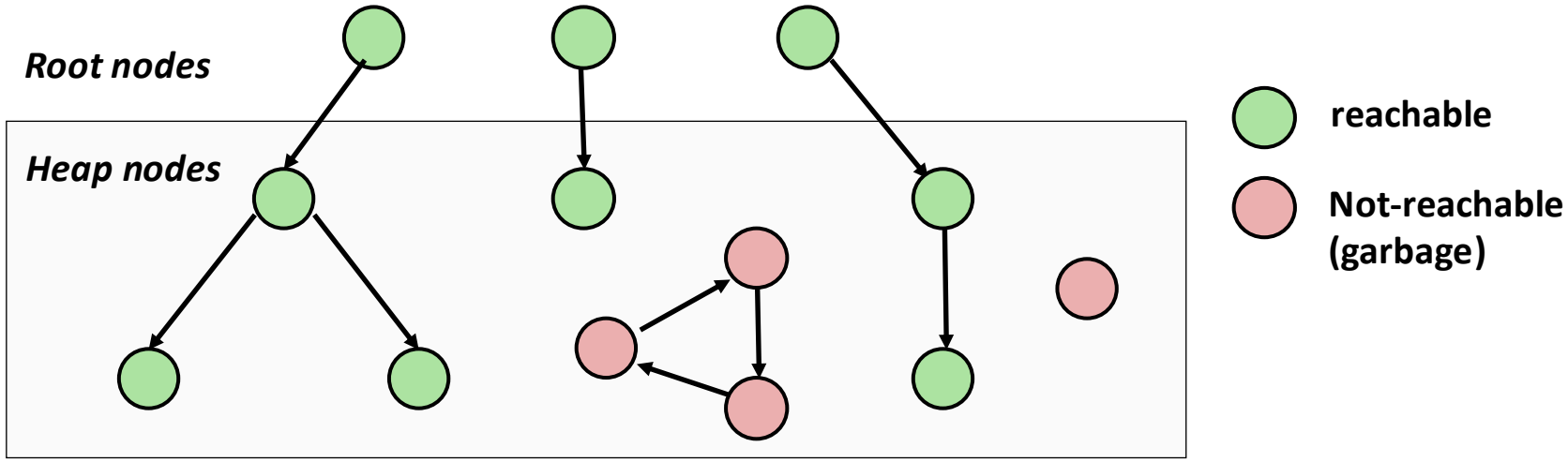
- Previously, we discussed explicit dynamic memory allocation
  - User programs use `malloc/free` to request memory space
- Some programming languages use implicit memory management
  - E.g., Java, Python, Perl, Scala, …
  - So that programmers are freed from memory-related bugs!
- Garbage collection: automatic reclamation of heap-allocated storage
  - A memory manager runs along with the user program
  - When a memory space is not needed, the memory manager frees it
  - User programs never need to free allocated memory!

Garbage Collection (GC)

- Key question: how does the memory manager know when a memory space can be freed?
- In general we cannot know what is going to be used in the future
- But we can tell whether a memory object can never be used any more, if there are no pointers anywhere that point to it
  - We may miss some opportunities to reclaim memory, but anything we reclaim is guaranteed to be not useful; correctness is always maintained
- To do so, language needs to ensure some rules ("*pointer is a pointer*")
  - We can distinguish between pointers and non-pointers
  - Any pointer only points to the start of an object
  - Cannot hide pointers, e.g., cast a `int*` into `long`, and then cast back

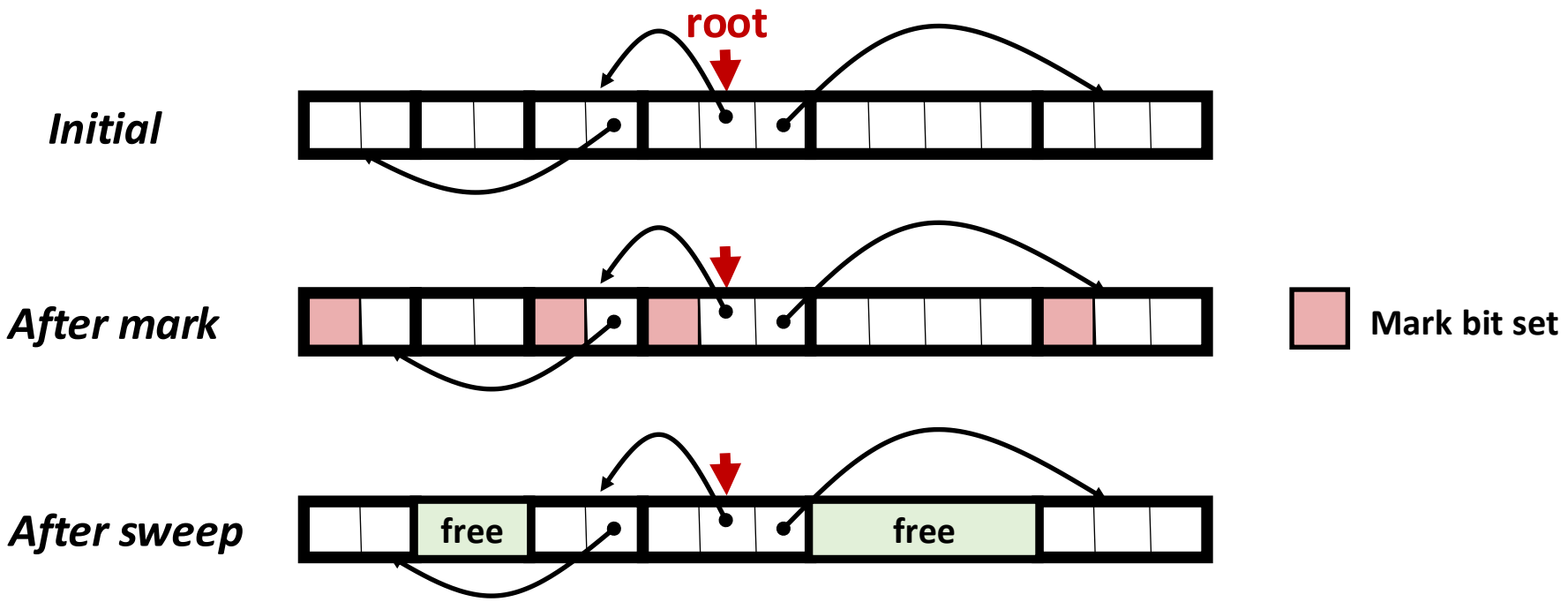
Classical GC Algorithm: Mark-and-Sweep

- View memory as a graph, where each each object is a node
  - An edge exists if a pointer in an object points to another object
- Root nodes: objects that are not dynamically allocated in the heap
  - E.g., registers, locations on the stack, global variables, …
- Reachable nodes: heap objects that are reachable from any root node
- Non-reachable nodes are garbage to be collected



Classical GC Algorithm 1: Mark-and-Sweep

- GC may start due to out-of-memory, or is periodically triggered
- Mark: start from root nodes, traverse graph and mark a bit in each reachable object; non-reachable objects are not visited and not marked
- Sweep: scan all objects in the memory, and free unmarked objects



Classical GC Algorithm 2: Reference Counting

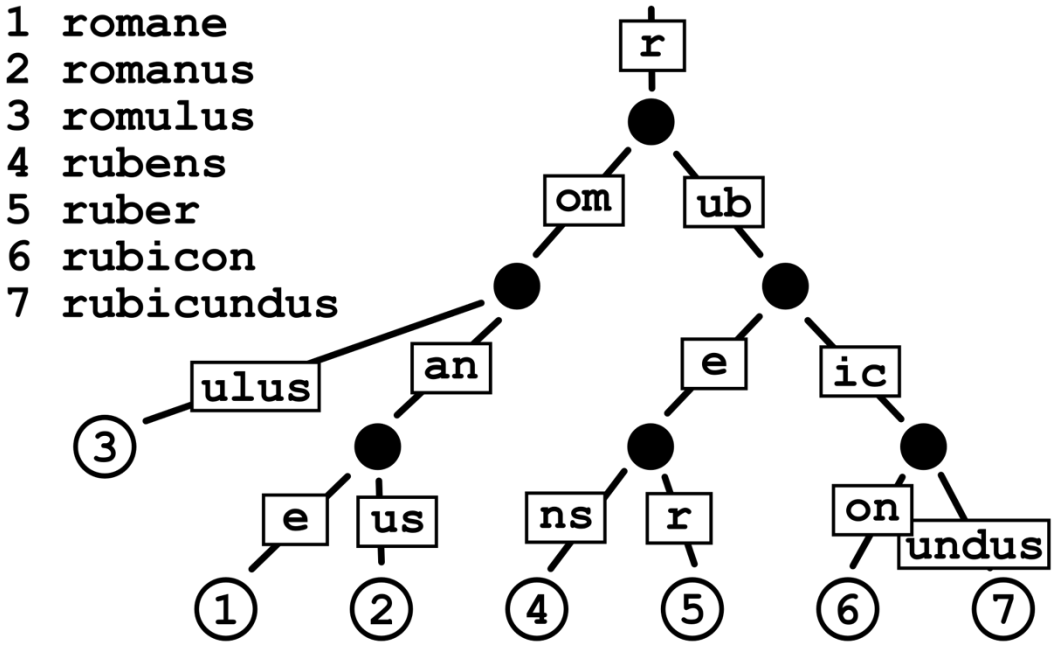
- Each object maintains a reference counter, which keeps the number of other pointers that point to this object
  - When assigning object to a pointer, increment reference counter of the object
  - When overwriting or deconstructing a pointer, decrement ref count of old object
- When the reference counter becomes zero, we free the object
- Problem: does not work for circular cases
  - E.g., A points to B, B points to C, C points to A, but with no external pointers
  - There are special ways to deal with this
- Tradeoffs
  - Mark-and-sweep delays memory reclamation, and may temporarily pause program execution
  - Reference counting is simple, frees objects more timely, has small impact on normal program execution; but may leak memory

Next Lecture:
I/O and Storage

Appendix

Radix Tree

- At each tree node, use different chunks of characters/bits in the key to branch
  - Char/bit chunks at each tree level may be the same or different lengths
  - In multi-level page tables, bit length is fixed
- Keys are implicitly represented as paths from root to leaves
  - Tree height depends on key length, not on number of elements
  - Only existing keys need paths; nodes of non-existing keys can be eliminated
- No rebalancing required



Why GC? – Common Memory-Related Bugs

- Dereference bad pointers

```
int val;
scanf("%d", val);
```

- Allocate wrong size

```
int **p;
p = malloc(N * sizeof(int));
for (int i = 0; i < N; i++) {
   p[i] = malloc(M * sizeof(int));
}
```

- Write unallocated memory (off-by-1)

```
int *p;
p = malloc(N * sizeof(int));
for (int i = 0; i <= N; i++) p[i] = i;
```

- Not checking the max length

```
char s[8];
gets(s);  // read 12345678 from input
```

- Misunderstanding pointer arithmetic

```
int *p;
while (p && *p != val) p += sizeof(int);
```

Common Memory-Related Bugs (Cont')

- Reference non-existent variables

```
int* foo() {
  int val;
  return &val;
}
```

- Reference freed space

```
int* x = malloc(N * sizeof(int));
...
free(x);
int* y = malloc(N * sizeof(int));
for (int i = 0; i < N; i++) y[i] = x[i];
```

- Forget to free
  - Slow eat memory space

```
void foo() {
  int* x = malloc(N * sizeof(int));
  ...
  return;
}
```

- Double free
  - E.g., copy-paste

```
int* x = malloc(N * sizeof(int));
...
free(x);
int* y = malloc(M * sizeof(int));
...
free(x);
```